SDK MANUAL

Neeuro OS SDK for Unity3D
Version S008

Maekell Ivan Mina

Created: 7th June 2024

Contents

(A) About

The NEEURO SenzeBand SDK is for developers who want to use the NEEURO SenzeBand to create apps or games that incorporate EEG technology.
Neeuro SenzeBand can be connected by using Bluetooth Low Energy (BLE) connection.
In Neeuro SenzeBand SDK Standard version, the SDK will analyze the data received and provide the following output to be used in the application.
1. Mental State - Attention classification
2. Mental State - Relaxation classification
3. Mental State - Workload classification
4. Oxygen Level (SPO2) and Heart Rate
5. Accelerometer values - X, Y, Z axes
6. Battery level
7. Channel signal status - (T/F) strong EEG signal detected on 4 channels
8. Signal ready status - (T/F) sufficient strong EEG signal received for signal processing for Mental States
9. Connection status - (T/F) Good connection on BLE, no data lost
10. Frequency Band Data
11. Raw and Filtered EEG Data
12. Raw PPG Data

Please enter the provided Developer Code in the NSB_Manager game object. The AuthenticationResult delegate will receive a True value when authentication succeeds. Please also ensure your device has an internet connection.

(B) System Requirements

- Unity 2022.2.4f1 or newer
- SenzeBand device
- Developer Code (to be issued with SDK)
- Bluetooth 5.0 on build devices

Addtional requirements for Windows build
- Bluetooth connection capability (can use Bluetooth dongle)

Additional requirements for Android build
- Android SDK at least API level 26 (preferably latest version)
- Gradle 7.5.1 (download here: https://gradle.org/releases/)

Devices unsupported :
- iPad 2
- Samsung Tab A(2016) 7inch
- Some computers' Bluetooth hardware may not be able to support the data transfer rate

(C) Package contents

“Neeuro_SenzeBand_Example_Project” - Sample Unity Project - Contains a SenzeBand control panel scene to show what the NEEURO SenzeBand can do.
It contains code with comments on how to use the SDK.
Uses Unity 2022.2.4f1

(D) SenzeBand Features and Use Info

The SenzeBand's sensors has a sampling rate of 250Hz, ie. each sensor produces 250 sampling values in 1 second.

The SenzeBand's Bluetooth transfer rate with the smart device is 1Hz, ie. the SenzeBand sends 1 data packet to the smart device every second.

The data packet contains raw data for 4 sensors, hence 4 sets of 250 values. Each set holds the signal values at every 1/250 seconds.

The accelerometer values also come at 1 Hz, ie. 1 sample in a data packet every second.

(E) Setup for Project

Use Unity 2022.2.4f1 or later

From SDK Unity Package:

1. Start by importing the asset package into your scene.
2. Make sure that the prefab "NSB_Objs" is in your scene. The prefabs are in Project window > Assets > NSB_SDK > Prefabs. The "NSB_Objs" prefab contains the three essential NSB prefabs inside namely NSB_BLE, NSB_EEG, and NSB_Manager (keep these three objects disabled). These are required to be able to communicate with the SenzeBand plugin. The ForceAskPermission and PermissionCanvas will be used for triggering permission popups for Android and IOS devices so the app can use Bluetooth services. ForceAskPermission will be in charge of enabling the NSB_BLE, NSB_EEG, and NSB_Manager objects after bluetooth services are allowed for the app. Feel free to edit the EnablePermissionsPopup_Android and EnablePermissionsPopup_iOS according to your app design and identification.
3. Check that NSBAndroidStandard.dll, NSBIOSStandard.dll, NSBWindowsStandard.dll and SenzeBandWindowsIPC_ClientDLL.dll are inside folder /Assets/DLL
4. Check that NSBAndroidUnityPlugin-standard-release.aar and AndroidManifest.xml are in folder /Assets/Plugins/Android
5. Check that IOSBLERobot.framework is in folder /Assets/Plugins/IOS
6. Check that win_res folder and win_res.zip is folder /Assets/
7. Under Unity Editor's Build Settings, switch your platform to Android, iOS or Windows.
8. You can now start developing using the SDK!

From Sample Unity Project
"demo" scene

1. Use Unity to open the project folder
2. Open "demo" scene
3. The GameObject NSB_Objs contain the required objects to communicate with plugin.
    a. The GameObject NSB_Manager is the main point to control and receive Mental State data. It communicates to plugin via NSB_EEG and NSB_BLE.

b. The ForceAskPermission is what enables bluetooth services for Android and iOS. It asks for permission from OS to utilize Bluetooth.

4. Under Build Settings, switch your platform to Android, iOS or Windows.
5. Follow the next set of instructions for building onto the platforms.

Building into devices:

For IOS

1. Under Build Settings -> Player Settings -> Other Settings -> API compatibility Level -> Make sure it is at .NET Framework
2. Build the Unity project into Xcode project
3. Open the Xcode project
4. Under "General -> Frameworks, Libraries, and Embedded Content", click Add (+) -> Add Other... -> Add Files... -> Frameworks -> Plugins -> IOS -> IOSBLERobot.framework
5. Under Capabilities -> On Background Mode, turn it on and select “Uses Bluetooth LE accessories”
6. Under Build Settings, search for "Runpath Search Paths" then change the value to "@executable_path/Frameworks"
7. Under Build Settings, search for “Bitcode” -> Build Options -> Enable Bitcode = “No”. Make sure the value is "No" for all levels.
8. Under Info -> iOS Target Properties, Add a property "Privacy - Bluetooth Always Usage Descriptor", give a value "For connection with Neeuro SenzeBand". Add a property "Privacy - Bluetooth Peripheral Usage Description", give the same value. Add a property "Privacy - Location When In Use Usage Description", give a value "For detecting nearby SenzeBand devices".
9. Remove “libiconv.2.dylib” on the Project panel
10. Under Build Phases, drag Embed Frameworks to the 3rd position.
11. Build and Run on Xcode, and the connected iOS devices

For Android

1. Under Build Settings -> Player Settings -> Other Settings -> set Minimum API Level to Android 8.0 'Oreo' (API level 26).
2. Under Build Settings -> Player Settings -> Other Settings -> Scripting Backend -> make sure that it is at IL2CPP. Include ARM64 in Target Architectures.
3. Under Build Settings -> Player Settings -> Other Settings -> API Compatibility level -> Make sure that it is at .NET Framework. Set Mute Other Audio Sources to TRUE.
4. To be able to build, we need to set the Gradle version to 7.5.1. Go to Edit -> Preferences -> Extenal Tools. Browse location of your downloaded Gradle. (e.g: \...\gradle-7.5.1-all\gradle-7.5.1)
5. Build the Unity project to generate APK file
6. The APK file can be copied to the Android devices, to install your app project

For Windows

1. Under Build Settings -> Player Settings -> Other Settings -> API Compatibility level -> Make sure that it is at .NET Framework
2. Build the Unity project to output Windows program. Note: WindowsBuild.cs Editor script will automatically copy win_res folder into the build folder for bluetooth process handling.
3. To manually copy win_res folder to the build, extract the "win_res.zip" file in the "Assets" folder to the same directory.

4. From "/Assets/win_res/" folder, copy ".server" folder to be in the same folder as the output EXE file. This program is required for Windows Bluetooth connection to Neeuro SenzeBand.

(F) Interface

View html documentation for more info.

Reference on data received from SDK

Functions in NSB_Manager.cs related to data received/processed from SenzeBand

```
public bool IsBluetoothEnabled()
    /// boolean value stating whether Bluetooth is enabled or not

public bool IsScanning()
    /// boolean value stating whether the app is scanning for available SenzeBand devices or not

public void SetScanning(bool state)
    /// Turns ON or OFF scanning for SenzeBand device
    /// param name="state" is scanning state ON or OFF

public List<string> GetScannedSenzeBandList()
    /// List of available SB devices found from scanning

public bool ConnectSB(string address)
    /// Starts connection process to SenzeBand device
    /// param name="address" is Address of device to connect to
    /// returns success or failure of connection attempt

public void sendCommand(string command)
    /// sends specific commands to plugin
    /// possible string commands:
    /// "COMMAND_START" - for starting EEG
    /// "COMMAND_STOP" - for stopping EEG
    /// "COMMAND_AC_LEADOFF" - for turning ON impedance check mode
    /// "COMMAND_DC_LEADOFF" - for turning OFF impedance check mode
    /// "COMMAND_LIGHT_RED" - sets SenzeBand device light to red color
    /// "COMMAND_LIGHT_GREEN" - sets SenzeBand device light to green color
    /// "COMMAND_LIGHT_BLUE" - sets SenzeBand device light to blue color
    /// "COMMAND_LIGHT_CYAN" - sets SenzeBand device light to cyan color
    /// "COMMAND_LIGHT_MAGENTA" - sets SenzeBand device light to magenta color
    /// "COMMAND_LIGHT_YELLOW" - sets SenzeBand device light to yellow color
    /// "COMMAND_STOP_RGB" - stops light commands
    /// "COMMAND_FW_VER" - command for getting the firmware version of SenzeBand device
    /// "COMMAND_CAL_START" - start calibration for acceleration and orientation to improve
accuracy
    /// "COMMAND_CAL_STOP" - stop calibration
```

```
public void DisconnectSB()
    /// Disconnects a connected SenzeBand device

public void AuthenticateUser()
    /// Tries to authenticate the SDK. Requires and internet connection

public int GetConnectionState()
    /// Determines the state of the NSB connection handling.
    /// Connection state: 0 - Not Connected, 1 - Connecting, 2 - Connected (partial), 3 - Connected
(no MCUID), 4 - Connected (Full with MCUID)

public string GetConnectedSBAddress()
    /// Determines address of connected SenzeBand device
    /// Bluetooth identified address of the connected SenzeBand device

public string GetConnectedSBMCUID()
    /// MCUID is a 32 character string unique to every SenzeBand device.
    /// This can be retrieved only after connection

public bool GetReceiveEEGState()
    /// Determines whether system is receiving EEG data from SenzeBand
    /// TRUE means that the NSB library sytem is receiving EEG and other data from the SB device,
FALSE if data transfer is switched OFF.

public void SetReceiveEEG(bool send)
    /// Sets the NSB library system to enable or disable receiving EEG and other data from the SB
device
    /// param name="send" is sending state ON or OFF

public bool GetReceivePPGState()
    /// Determines whether system is receiving PPG data from SenzeBand
    /// TRUE means that the system is receiving PPG data from SB device, FALSE if PPG data
transfer is switched OFF.

public void SetReceivePPG(bool send)
    /// Sets the NSB library system to enable or disable receiving PPG and other data from the SB
device
    /// param name="send" is sending state ON or OFF

public void SetGammaThreshold(float threshold)
    /// Only for SenzeBand 1, not applicable for SenzeBand 2
    /// Sets gamma threshold for filtering useful data
    /// param name="threshold" is set limit for what power is considered high interference.

public void Set5060Threshold(float threshold)
    /// Only for SenzeBand 1, not applicable for SenzeBand 2
    /// Sets 50-60Hz threshold for filtering useful data
    /// param name="threshold" is set limit for what power is considered high interference.

public float GetFiftySixtyReading(int channel)
```

```
    /// Only for SenzeBand 1, not applicable for SenzeBand 2
    /// Determines the 50-60Hz noise signal - This comes from power sources.
    /// parame name="channel" is the channel index to read 50-60Hz power from

public float GetGammaReading(int channel)
    /// Only for SenzeBand 1, not applicable for SenzeBand 2
    /// Determines frequency signal power - This is present in naturally-occuring ambient noise
    /// parame name="channel" is the channel index to read 50-60Hz power from

public float GetMeanReading(int channel)
    /// Only for SenzeBand 1, not applicable for SenzeBand 2
    /// Determines average signal power
    /// parame name="channel" is the channel index to read mean power from

public int GetAccel(int dimension)
//Returns the accelerometer values. Range from -2048 to 2048.
//Parameter: dimension: 0 - X axis, 1 - Y axis, 2 - Z axis

public float GetAttention()
//Returns the attention level value, range of 0 (low attention) - 1 (high attention).
//This is calculated from the latest set of EEG data received from the SB device.

public float GetRelaxation()
//Returns the relaxation level value, range of 0 (very tensed) - 1 (very relaxed).
//This is calculated from the latest set of EEG data received from the SB device.

public float GetMentalWL()
    // Returns the mental workload level value, range of 0 (not taxing) - 1 (very taxing).
// This is calculated from the latest set of EEG data received from the SB device.

public bool GetChannelStatus(int channel)
/// Returns if the signal from each sensor on SB device is receiving EEG signal
        /// Parameter: channel:
        ///     0=center-left
        ///     1=center-right
        ///     2=right
        ///     3=left

public bool GetSignalReady()
//Returns if the EEG signal received is acceptable.
//Signal noise from body movement, or insufficient skin contact at the sensor electrode give poor
quality signal, the results from signal processing may not be accurate.

public bool GetGoodBTConnection()
//Returns if the Bluetooth connection is good.
//Interference from other Bluetooth signal emitters can lead to data loss

public bool GetAuthenticationResult()
    /// Returns validity of authentication

public string GetAuthenticationStatus()
```

```
    /// Returns readable string reflecting auth status

public string GetConnectedSBBattery()
    /// Returns the battery level, ranging from 0 to 1

public float GetFrequencyBand(int channel, int band)
    /// Returns received frequency band data on selected channel and band
    ///This is calculated from the latest set of EEG data received from the SB device.
    /// param name="channel" is index of SenzeBand channel: 0 = center-left, 1 = center-right, 2 =
right, 3 = left
    /// param name="band" is index of frequency band: 0 = delta, 1 = theta, 2 = alpha, 3 = beta, 4 =
gamma

public float[] GetRawEEG()
    /// Returns received raw unprocessed EEG data, array of EEG data for all channels (0 to 249 =
center-left, 250 to 499 = center-right, 500 to 749 = right, 750 to 999 = left

public float[] GetFilteredEEG()
    /// Returns received cleaned up processed EEG data, array of EEG data for all channels (0 to 249
= center-left, 250 to 499 = center-right, 500 to 749 = right, 750 to 999 = left

public float[] GetEEGImpedance()
    /// Returns received EEG impedance
    /// This helps to verify if the signal quality receiving from the 4 contact points/electrodes are
good.
    /// Example, if there is makeup or some other impurities on the skin, this impedance values will
be much higher,
    /// due to the contribution of more resistance between the electrodes and skin surface, and the
EEG signals will not be good/accurate.
    /// Good impedance range is below 800kohms

public float[] GetCalibrationParameters()
    /// Returns received calibration parameters
    /// For calibrating acceleration and orientation to improve accuracy

public int GetSPO2()
    /// Returns received SPO2 data from plugin

public int GetHeartRate()
    /// Returns received heart rate data from plugin

public string GetConnectedSBVersion()
    /// Returns current connected SenzeBand version
```

(G) Support contact

For support, please email to support@neeuro.com